

Programozás II.

9. labor

Szekeres György

2026. április 19.

- 1 Előző órán...
- 2 Absztrakt és virtuális metódusok
- 3 List<T>
- 4 Összetett listák
- 5 Öröklődéses feladat – Hajók

Az előző órán tanultuk a(z)

- öröklődést.
- ZH írás.

- **abstract**: absztrakt metódus vagy osztály
 - Az absztrakt osztályból **nem lehet példányt létrehozni**
 - Az absztrakt metódusnak **nincs törzse**, csak aláírása
 - A leszármazott osztályoknak **kötelességük felüldefiniálni**
- **virtual**: virtuális metódus
 - Van alapértelmezett implementációja
 - A leszármazott osztály **felüldefiniálhatja** az override kulcsszóval
 - Nem kötelező felüldefiniálni

- Szintaxis:

```
1 abstract class Allat {
2     public abstract void HangotAd();    // nincs törzs
3 }
4
5 class Kutya : Allat {
6     public override void HangotAd() {
7         Console.WriteLine("Vau!");
8     }
9 }
10
11 class Allatkert {
12     public virtual void Etet() {
13         Console.WriteLine("Etetés folyamatban...");
14     }
15 }
```

- **Használat:**

```
1 Allat a = new Kutya();  
2 a.HangotAd();    // "Vau!"  
3  
4 Allatkert ak = new Allatkert();  
5 ak.Etet();       // alapértelmezett implementáció
```

List<T> alapok

- A List<T> egy dinamikus tömb C#-ban
- Tetszőleges típus tárolható benne (pl. int, string, saját osztály)
- Mérete futás közben változik
- Szintaxisa:

```
1 List<int> szamok = new List<int>();
```

- Elem hozzáadása:

```
1 szamok.Add(42);
```

- Elem elérése index alapján:

```
1 Console.WriteLine(szamok[0]);
```

- Elem törlése:

```
1 szamok.Remove(42);  
2 szamok.RemoveAt(0);
```

List<T> fontosabb metódusai

- **Count:** az elemek száma

```
1 int db = számok.Count;
```

- **Contains:** tartalmaz-e egy elemet

```
1 if (számok.Contains(42)) { ... }
```

- **Insert:** elem beszúrása adott indexre

```
1 számok.Insert(1, 99);
```

- **Clear:** lista kiürítése

```
1 számok.Clear();
```

- **Foreach bejárás:**

```
1 foreach (int szám in számok)
2     Console.WriteLine(szám);
```


List< List<T> >

- Többdimenziós, rugalmas szerkezet
- Gyakran használjuk mátrixok, táblázatok, grafok tárolására
- Szintaxisa:

```
1 List<List<int>> matrix = new List<List<int>>();
```

- Új sor hozzáadása:

```
1 matrix.Add(new List<int>() {1, 2, 3});
```

- Elem elérése:

```
1 int ertek = matrix[0][1];    // 2
```

- Elem módosítása:

```
1 matrix[0][1] = 99;
```

List< List<T> > bejárása

- Két egymásba ágyazott ciklussal járható be

```
1 foreach (var sor in matrix)
2 {
3     foreach (var elem in sor)
4     {
5         Console.Write(elem + " ");
6     }
7     Console.WriteLine();
8 }
```

- Tipikus felhasználás:
 - Mátrixműveletek
 - Játékpálya (grid) tárolása
 - Adattáblák kezelése

- A Bahart.csv egy sora:

```
1 8601340;Badacsony;katamarán;300000;220000;185000
```

- Absztrakt őszosztály: Hajo
 - ENI, név, típus, ár1–ár2–ár3
 - Minden adattag publikus
 - Paraméteres konstruktor
 - `abstract int BerletiDij(int orak);`
 - 1 óra: ár1
 - 2 óra: $2 * \text{ár2}$
 - 3+ óra: $\text{óra} * \text{ár3}$

- Leszármazott osztályok:
 - Katamaran
 - Nosztalgia
 - Szemely
 - Komp
- Beolvasás: `List<Hajo>` listába
- Bérleti díj számítása felhasználói input alapján

- Készítse el a `Flotta.cs`-t:
 - `List<Hajo>` tárolja a beolvasott hajókat
 - `Beolvas(string fajl)`: CSV beolvasása kivételkezeléssel
 - `Mutat()`: hajók neve és típusa
 - `AtlagAr(int orak)`: teljes flottára számolt átlag bérleti díj
 - `Legdragabb(int orak)`: visszaadja a legdrágább hajót
 - `Legolcsobb(int orak)`: visszaadja a legolcsóbb hajót
 - `SzuresTipusra(string tipus)`: kilistázza a megfelelő hajókat
 - `SzuresArra(int orak, int maxAr)`: hajók listázása ár alapján

- Kérje be a fájl elérhetőségét konzolról
- Példányosítsa a Flotta osztályt és olvassa be az adatokat
- Végtelen ciklusos menü készítése (állapotgép jelleggel)
 - Hajók listázása
 - Bérleti díj számítása
 - Legdrágább / legolcsóbb hajó keresése
 - Szűrés típusra
 - Szűrés árra